

Test Summary Report

Vaporant

Riferimento	VAPORANT_TSR
Versione	1.0
Data	//
Destinatario	Prof.re Andrea De Lucia
Presentato da	• Gianfranco Barba • Francesco Corcione
Approvato da	//

Revision History

Data	Versione	Descrizione	Autori
2026-06-13	1.0	Prima stesura. Definizione ed esecuzione della suite di unità sulle 6 classi del service layer (107 casi totali) tramite tool Maven.	Gianfranco Barba, Francesco Corcione
2026-07-15	1.1	Revisione finale	Gianfranco Barba, Francesco Corcione

Sommario

Revision History	2
1 Introduzione	4
2 Ambiente di esecuzione.....	4
3 Legenda esiti	5
4 Risultati globali (it.unisa).....	5
4.1 Test Summary	5
4.2 Packages	5
5 Dettaglio classe per classe (it.unisa.service)	6
5.1 Package it.unisa.service	6
5.1.1 Classes Summary	6
6 Analisi delle anomalie e gap di copertura	6
7 Conclusioni e considerazioni finali	7

1 Introduzione

Il presente documento costituisce il **Test Summary Report (TSR)** relativo alle attività di verifica dinamica condotte a livello di unità (*unit testing*) sul package `it.unisa.service` del sistema software **Vaporant**. Tale package incapsula l'intera logica di business introdotta a valle della manutenzione perfetta (re-engineering architetturale) finalizzata all'adozione del pattern Service-oriented e al disaccoppiamento dei componenti legacy.

L'obiettivo primario dell'attività risiede nella certificazione formale della correttezza logico-funzionale del livello di servizio. Al fine di garantire l'indipendenza dei test dalle strutture persistenti del database e dai componenti del livello di accesso ai dati (DAO), è stato implementato un isolamento rigoroso delle classi target mediante la definizione di controfigure di test (*test doubles*) per mezzo del framework **Mockito**.

Ove applicabile (es. metodi di validazione delle classi `UserService` e `AddressService`), la progettazione dei test case ha seguito i criteri formali del **Category Partition** (identificazione delle classi di equivalenza valide e invalide) e della **Boundary Value Analysis (BVA)** sui campi a lunghezza fissa, applicando il principio dell'isolamento dei difetti (variazione di un singolo parametro non valido per volta).

Tutti i risultati documentati derivano dall'esecuzione automatizzata della suite mediante la CLI di build. I comportamenti anomali riscontrati (descritti nella Sezione 6) sono stati volutamente preservati in conformità al vincolo metodologico di non-modifica del codice di produzione durante la campagna di test.

2 Ambiente di esecuzione

La suite di test è stata eseguita in un ambiente locale conforme alle specifiche di deploy del progetto:

- **JDK:** Versione 17.0 (target di compilazione del sistema).
- **Testing Framework:** JUnit Jupiter 5.10.2 (JUnit 5) integrato con Mockito JUnit Extension (versione 5.11.0) per la gestione del ciclo di vita dei mock.
- **Build Automation & Execution:** Apache Maven (tramite il plugin `maven-surefire-plugin` per il discovery e l'esecuzione delle classi conformi al pattern `*Test.java`).
- **Isolamento architetturale:** Nessuna dipendenza esterna attiva (database offline, server applicativi spenti). I componenti DAO associati ai servizi sono mockati per garantire che l'esecuzione dei test eserciti unicamente i cammini logici del service layer.

- **Organizzazione sorgenti:** Le classi di test risiedono nella directory dedicata test/java, parallela alla cartella src, per prevenire conflitti in fase di compilazione ordinaria dell'applicazione.

3 Legenda esiti

Gli esiti delle asserzioni sono determinati dal framework JUnit e formalizzati come segue:

- **Passed:** Il comportamento osservato coincide con l'oracolo di test predefinito; il cammino d'esecuzione si è concluso senza violazioni.
- **Failed:** È stata rilevata una divergenza tra l'output reale e l'oracolo atteso, indicando un'anomalia logica nella classe sotto test. A questo livello di astrazione non viene applicata la tassonomia tra anomalie Funzionali (F) e Non Funzionali (NF).

4 Risultati globali (it.unisa)

4.1 Test Summary

Totale Test	Falliti	Ignorati	Durata complessiva	Percentuale successo
117	0	0	0.535s	100%

4.2 Packages

Package	Test	Falliti	Ignorati	Durata	Successo
it.unisa.service	117	0	0	0.535s	100%

5 Dettaglio classe per classe (it.unisa.service)

5.1 Package it.unisa.service

5.1.1 Classes Summary

Classe di test	Test eseguiti	Falliti	Ignorati	Durata	Percentuale successo
AddressServiceTest	28	0	0	0.025s	100%
CartServiceTest	10	0	0	0.010s	100%
InvoiceServiceTest	6	0	0	0.005s	100%
OrderServiceTest	3	0	0	0.005s	100%
ProductServiceTest	28	0	0	0.040s	100%
UserServiceTest	42	0	0	0.450s	100%

6 Analisi delle anomalie e gap di copertura

La campagna di verifica non ha rivelato comportamenti impreveduti al di fuori di quelli già tracciati come limitazioni note o incident aperti nel sistema di tracciamento delle anomalie (Test Incident Report):

- UserService - Tolleranza input vuoti (TC_2.1_11 / INC-06):** Il test case `isValidRegistration_codFSoliSpaziLunghezza16` si conclude con esito *Passed* pur validando un input errato (codice fiscale composto da soli spazi bianchi). Questo esito certifica la conformità del test rispetto al comportamento reale del metodo legacy, che effettua unicamente un controllo quantitativo di lunghezza (`length == 16`) senza validare l'espressione regolare del formato. Il difetto è formalmente aperto e documentato nel TIR.
- OrderService - Limitazione architetturale sul checkout:** La procedura `checkout()` richiede la risoluzione dinamica delle risorse di connessione JDBC tramite lookup JNDI gestito direttamente dal modulo servlet container (Apache Tomcat). Non essendo disponibile tale contesto all'interno della JVM isolata durante l'esecuzione dei test di unità Maven, e al fine di evitare modifiche intrusive al codice sorgente per l'introduzione di dependency injection custom, si è scelto di escludere il

metodo dalla suite JUnit. La copertura del requisito funzionale associato è delegata interamente alla suite di collaudo di sistema (Katalon Studio) in ambiente di staging containerizzato.

7 Conclusioni e considerazioni finali

L'esecuzione della suite di unit testing ha consentito di verificare le 6 classi del service layer, garantendo una copertura logica estesa delle condizioni di input e delle eccezioni propagate dai DAO.

L'integrazione del framework Mockito ha permesso di validare la correttezza algoritmica di servizi cruciali (come il calcolo delle imposte in InvoiceService, la crittografia degli hash delle credenziali in UserService e i filtri di catalogazione in ProductService) indipendentemente dallo stato dei dati persistenti.

Il completamento di questa campagna con il 100% di esiti positivi (117/117 test passed) certifica l'efficacia del processo di re-engineering e garantisce che la logica di business introdotta con le modifiche perfettive sia solida, testabile ed isolata dal debito tecnologico del codice legacy preesistente.